

Documentación oficial

Meta Cloud API

Guía de configuración para WhatsApp Business oficial

De qué trata este documento

Paso a paso para conectar tu número de WhatsApp a Meta Cloud API (proveedor oficial). Incluye creación de app en Meta Developer Portal, obtención de tokens, configuración de webhooks y validación.

FastChat DJ · Plataforma WhatsApp CRM con IA

Documento 01 · Generado desde `docs/meta_setup.md`

Última actualización: abril 2026

Contenido

1.Arquitectura en una línea

2.1. En Meta (lado cliente)

3.2. En FastChat

4.3. Configurar el webhook en Meta

5.4. Plantillas (obligatorias para conversaciones nuevas)

6.Debug

- A. Verificar en Django Admin

- B. Probar envío manualmente

- C. Logs típicos

- D. cURL de prueba

- E. Errores comunes

7.Modelos involucrados

8.Endpoints

9.Lo que NO requiere Meta (a diferencia de Baileys)

Pasos para conectar una sesión FastChat al transporte oficial de WhatsApp (Meta Cloud API). Apunta tanto al setup inicial como al debug típico.

Arquitectura en una línea

`SesionWhatsApp.proveedor='meta'` activa una `ConfigMeta` (OneToOne) con las credenciales de Graph API. El dispatcher `get_whatsapp_service(sesion)` rutea los envíos a `MetaWhatsAppService`; el receiver `/whatsapp/meta_webhook/` traduce los eventos al formato interno y los pasa al mismo `process_incoming_message` que usa Baileys.

1. En Meta (lado cliente)

1. **Meta Business Manager** → crear una **WhatsApp Business Account (WABA)**.
2. Agregar (o portar) un **número de teléfono** a la WABA. Verifícalo por SMS/voz.
3. **Apps** → crear una app de tipo "Business" y agregar el producto "**WhatsApp**".
4. Copiar:
5. **WABA ID** (ID de la cuenta WhatsApp Business)
6. **Phone Number ID** (ID del número específico — es el routing ID que usa la API para saber por qué número enviar)
7. **Business Account ID** (opcional — el ID de la cuenta de negocio que contiene la WABA)
8. **System Users** → crear un *system user* con permiso sobre la WABA y generar un **access token permanente** (no caduca).
9. App → Settings → Basic → copiar **App ID** y **App Secret** (este último se usa para validar la firma HMAC de los webhooks).

2. En FastChat

1. Login → **Sesiones WhatsApp** → "Nueva sesión Meta".
2. En el form de la sesión, sección **Configuración Meta**, pegar:
3. WABA ID
4. Phone Number ID
5. Access Token (permanente)
6. App Secret (necesario para validar HMAC del webhook)
7. Guardar. El sistema:

8. Marca `proveedor='meta'`.
9. Genera un `webhook_verify_token` aleatorio.
10. Muestra la **URL del webhook** que debes configurar en Meta:
`https://<TU_DOMINIO>/whatsapp/meta_webhook/`
11. Click en "Verificar conexión Meta" → llama a Graph API y rellena `display_phone_number`, `quality_rating` y `messaging_limit_tier`.

3. Configurar el webhook en Meta

Meta → App → WhatsApp → Configuration → Webhooks:

- **Callback URL:** `https://<TU_DOMINIO>/whatsapp/meta_webhook/`
- **Verify Token:** copiar el que generó FastChat (visible en el form de la sesión).
- Suscribir al menos los campos: `messages`, `message_template_status_update`, `account_update`.

Al guardar, Meta hace un **GET** al endpoint con `hub.challenge`. Si el verify token coincide, el handshake pasa y FastChat marca `ConfigMeta.webhook_verificado_en = now()`.

4. Plantillas (obligatorias para conversaciones nuevas)

Fuera de la **ventana de servicio de 24h** (24h después del último mensaje del cliente), Meta solo permite enviar **plantillas pre-aprobadas**.

En FastChat: **Plantillas WhatsApp** → crear → someter a Meta. Estados: `BORRADOR` → `PENDING` → `APPROVED|REJECTED`. Meta tarda entre minutos y 24h en revisar. Sincroniza con el botón "**Sincronizar**" para refrescar el estado desde Graph API.

Debug

A. Verificar en Django Admin

`/admin/whatsapp/`:

- **ConfigMeta:** ver `webhook_verificado_en` (debe estar lleno), `quality_rating`, `messaging_limit_tier`, `ultima_sincronizacion`.
- **EventoMetaRecibido:** cada webhook que llega se guarda crudo. Filtra por `firma_valida=False` para detectar problemas de App Secret. Filtra por `procesado=False` o `error_procesamiento` para ver fallas.

- **PlantillaWhatsApp**: estado actual y cantidad de envíos.

B. Probar envío manualmente

En **Sesiones** → editar la sesión Meta → botón **"Probar envío"** envía un mensaje al número configurado (debe estar dentro de la ventana de 24h o ser una plantilla aprobada).

C. Logs típicos

```
Meta webhook verificado para ConfigMeta id=X (WABA Y) ← handshake OK
MetaService: sesion <id> no tiene ConfigMeta ← falta config
MetaService: HMAC invalida en webhook ← App Secret mal
```

D. cURL de prueba

```
# test handshake (debe responder con el challenge)
curl "https://<DOMINIO>/whatsapp/meta_webhook/?hub.mode=subscribe&hub.verify_token=<VERIFY_TOKEN>&hub.challenge="
# Verificar credenciales contra Graph API directo
curl -H "Authorization: Bearer <ACCESS_TOKEN>"
https://graph.facebook.com/v21.0/<PHONE_NUMBER_ID>?fields=display_phone_number,quality_rating
```

E. Errores comunes

Síntoma	Causa probable
Webhook GET retorna 403	hub.verify_token no coincide con ConfigMeta.webhook_verify_token .
Webhook POST guarda evento con firma_valida=False	app_secret mal copiado o vacío.
send_text_message retorna error 401/403	Access token expirado o sin permiso sobre la WABA.
send_text_message retorna error 131047 ("re-engagement message")	Pasaron >24h del último mensaje del cliente. Usa una plantilla.
Plantilla queda en PENDING mucho tiempo	Normal; Meta puede tardar hasta 24h. Usa "Sincronizar".

Plantilla REJECTED	Revisar motivo_rechazo en el admin. Causas típicas: contenido promocional en categoría UTILITY, variables sin ejemplo, etc.
---------------------------	--

Modelos involucrados

- **whatsapp.SesionWhatsApp** — campo **proveedor** y propiedades **es_meta/es_baileys**.
- **whatsapp.ConfigMeta** — credenciales y estado por sesión.
- **whatsapp.PlantillaWhatsApp** — plantillas con su estado en Meta.
- **whatsapp.EventoMetaRecibido** — auditoría cruda de webhooks (read-only en admin).

Endpoints

Método	URL	Para qué
GET	/whatsapp/meta_webhook/	Handshake de verificación de Meta
POST	/whatsapp/meta_webhook/	Recepción de eventos firmados
POST	/api/enviar-mensaje/	Envío externo (rate-limited 30/min)

Lo que NO requiere Meta (a diferencia de Baileys)

- No hay **QR**: la sesión nace conectada cuando las credenciales son válidas.
- No hay **reconexión** automática (no hay socket persistente — el cron **reconectar_sesiones.py** ya filtra **proveedor='baileys'**).
- No hay **sync de contactos** ni **update de perfil** vía API: gestiónalo desde Meta Business Manager.
- No hay **mensajes programados** vía API libre: usa plantillas pre-aprobadas.